



An-Najah National University
Computer Engineering department
Distributed Operating Systems
Turning the Bazar into an Amazon
Design document

Aya Sawalha

Raheeq Qassrawi

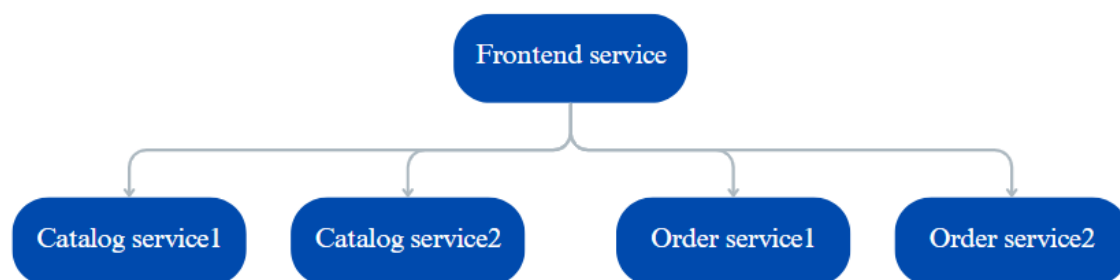
Each folder inside src folder can be considered as a service and can be replicated. And each of services has responsibilities:

- Catalog service: to get information about specific book or getting set of books of the same topic, in addition to store current stock of each book.
- Order service: to purchasing books.
- Frontend service: to send query and purchasing requests to other services.

In this project, we are required to achieve replication, caching and consistency.

Replication:

We achieved replication by running two containers of the same image, this is implemented in the docker-compose file, we created two order services running on ports 3001 and 3002, and other two catalog services running on ports 4001 and 4002.



Frontend service is not replicated, and is used to communicate with other services, when we request a specific book or request books by their topic, and these requests made from frontend service, it applies round robin algorithm for load balancing, which sends requests to one of the catalog services in a sequential manner.

Also, purchasing requests are not always sent to single service when they are sent from frontend service, we balanced the load in on order services in the same manner we used with catalog services.

Caching:

Caching is used for quick fetching data rather than requests. We store data in internal memory (we used map to create the in-memory cache). So, when user from any service (catalog1 or catalog2) query for book information or books by their topic we look for desired data whether it exists in the cache. If so, return data immediately without sending a request to get it. If not, send a request to get the data and store fetched data into the cache for the upcoming requests.

When data is changed, we invalidate the record (remove it from cache) so that the upcoming requests get the updated version of data.

Consistency:

Consistency means that when data is changed in a service, all replicas should be notified with that change. What we did to achieve consistency is when an update occurs to one of the services, we send this update to related services to update their corresponding record, after that we inform user that data has been updated successfully. By this we ensure that all replicas have identical copy of data.

For example, if an update comes to catalog1 service (to update quantity when purchasing for example) the update is sent to other replica, since each replica has an environment variable called REPLICA we use to know where this request is coming from, if the request comes from catalog1, then we send update request to catalog2 service with same requested date.

As we see when updating data in one service the effect is reflected to the other, and here there is a potential to get into an infinite loop of calling the other replica. To avoid this, we used a variable sent with the request called `internal` to notify the other service that this request is coming from parent request so do not notify the other replica and return back.

How it works?

As we need to run multi-services in a single application we used docker-compose to create 5 different containers: 1 frontend service, 2 catalog services and 2 order services.

For each service we specify the folder we need to run as a single container, and this folder contains its own Dockerfile that contains the instructions to build this image.

Docker-compose simplify the control of entire application, for each service we select the folder to build containers from using `build` property, we map the ports of container and the port exists in code using `ports` property, for catalog and order replicas we added environment variable called `REPLICA` to specify where this request comes from using environment variables, we created `volumes` to match files synchronize writing data on files outside container's file system.

We created single network called `proj-net` for the whole application .

To run the application, we can either clone it from GitHub or download it as a .zip file then open it and build is using `docker-compose up --build` instruction. Make sure that docker is running on the device before building the project.